Universidad Rafael Landívar

Ingeniería En informática y Sistemas

Inteligencia Artificial

Proyecto

Jhonatan Velasquez Fuentes

Carne: 1137521

Guatemala, 24 de abril del 2026

1. Introducción

En el presente proyecto se desarrolla un sistema de clasificación automática de solicitudes de soporte al cliente utilizando técnicas de procesamiento de lenguaje natural (NLP). El propósito principal es simular el funcionamiento de una mesa de ayuda inteligente capaz de enrutar tickets hacia su categoría correspondiente sin intervención humana.

Este tipo de sistemas resulta especialmente útil en entornos empresariales donde se manejan grandes volúmenes de solicitudes, permitiendo reducir tiempos de respuesta, minimizar errores humanos y optimizar recursos operativos.

El modelo implementado se basa en el algoritmo Naïve Bayes Multinomial, construido completamente desde cero sin el uso de librerías de aprendizaje automático de alto nivel. Adicionalmente, se desarrolló una aplicación web utilizando Flask que permite interactuar con el modelo en tiempo real.

2. Definición del Problema y Objetivos

2.1 Problema

Las organizaciones que brindan soporte al cliente reciben diariamente miles de solicitudes en formato de texto. Estas deben clasificarse manualmente en categorías como órdenes, facturación, envíos o reembolsos. Este proceso presenta varias desventajas:

- Alto consumo de tiempo
- Dependencia de recursos humanos
- Propensión a errores
- Dificultad para escalar

2.2 Objetivo General

Desarrollar un sistema automatizado de clasificación de tickets de soporte utilizando el algoritmo Naïve Bayes implementado desde cero.

2.3 Objetivos Específicos

- Aplicar técnicas de preprocesamiento de texto
- Implementar el modelo Naïve Bayes Multinomial
- Representar texto mediante Bag of Words
- Incorporar Laplace Smoothing para manejo de probabilidades
- Utilizar logaritmos para estabilidad numérica
- Evaluar el modelo con validación cruzada (K-Folds)
- Desarrollar una interfaz web para predicción en tiempo real

3. Descripción del Dataset

Se utilizó el dataset Bitext Customer Support LLM Chatbot Training Dataset, el cual contiene más de 26,000 registros etiquetados.

Características principales:

- Variable de entrada: instruction (texto de la solicitud)
- Variable objetivo: intent (categoría)
- Total, de categorías: 11

Ejemplos de categorías:

- cancel_order
- change_order
- get_refund
- place_order
- complaint
- track_order

Este dataset es adecuado para clasificación supervisada debido a su consistencia semántica entre texto y etiquetas.

4. Preprocesamiento de Datos

El preprocesamiento es una etapa crítica para mejorar el desempeño del modelo. Se realizaron las siguientes transformaciones:

- Conversión a minúsculas
- Eliminación de caracteres especiales
- Tokenización (división del texto en palabras)
- Eliminación de stopwords (palabras sin carga semántica relevante)

Representación: Bag of Words

Cada documento se transforma en un vector numérico que representa la frecuencia de aparición de cada palabra dentro de un vocabulario global.

Formalmente, si el vocabulario tiene tamaño |V|, cada documento se representa como un vector:

$$X=(x1,x2,...,x|V|)$$

donde xi indica la frecuencia de la palabra i en el documento.

5. Algoritmo Naïve Bayes

5.1 Fundamento Teórico

El modelo se basa en el Teorema de Bayes, el cual permite calcular la probabilidad posterior de una clase dada una observación:

$$P(Ck|X)=P(X)P(X|Ck)/P(Ck)$$

Donde:

- P(Ck|X): probabilidad de la clase dada el documento
- P(X|Ck): verosimilitud del documento dada la clase
- P(Ck): probabilidad a priori de la clase
- P(X): probabilidad del documento (constante)

En clasificación, se selecciona la clase que maximiza:

$$C\hat{}=\arg Ck \max P(X|Ck)\cdot P(Ck)$$

5.2 Suposición de Independencia (Naïve)

El modelo asume que las palabras son **condicionalmente independientes** dado la clase:

$$\mathbf{P(X \mid Ck)} = \prod_{\mathbf{i=1}}^{n} P(xi \mid Ck)$$

Esta simplificación reduce significativamente la complejidad computacional, aunque no siempre refleja la realidad lingüística.

5.3 Modelo Multinomial

En este enfoque, se considera la frecuencia de palabras:

$$\mathbf{P(X \mid Ck)} = \prod_{\mathbf{i=1}}^{|\mathbf{V}|} P(wi \mid Ck)^{xi}$$

donde:

- P(wi|Ck): probabilidad de la palabra wi en la clase Ck
- xi: frecuencia de la palabra en el documento

5.4 Laplace Smoothing

Uno de los principales problemas del modelo es la probabilidad cero. Si una palabra no aparece en una clase durante el entrenamiento, entonces:

$$P(wi|Ck)=0$$

Esto provoca que toda la probabilidad del documento sea cero.

Para evitarlo, se utiliza Laplace Smoothing (suavizado aditivo):

$$(\mathbf{wi} \mid \mathbf{Ck}) = \frac{\mathbf{count}(\mathbf{wi}, \mathbf{Ck}) + \mathbf{1}}{\sum \mathbf{jcount}(\mathbf{wj}, \mathbf{Ck}) + \mid \mathbf{V} \mid}$$

Donde:

- $\mathrm{count}(\mathrm{wi}, \mathrm{Ck})$: número de veces que aparece la palabra en la clase
- |V|: tamaño del vocabulario

Este ajuste garantiza que ninguna probabilidad sea cero y mejora la generalización del modelo.

5.5 Uso de Logaritmos

El producto de muchas probabilidades pequeñas puede generar problemas de **underflow numérico**.

Para solucionarlo, se aplica logaritmo:

$$\mathbf{log}\,\mathbf{P}(\mathbf{X} \mid \mathbf{Ck}) = \prod_{\mathbf{i=1}}^{|\mathbf{V}|} xi * \log P(wi \mid Ck)$$

Esto convierte productos en sumas, mejorando la estabilidad computacional.

6. Métricas de Evaluación y Matriz de Confusión

6.1 Matriz de Confusión

La matriz de confusión es una representación tabular que permite evaluar el desempeño de un modelo de clasificación multiclase comparando las predicciones con los valores reales.

Formalmente, es una matriz M de tamaño n×n, donde:

- n: número de clases
- Mij: número de instancias de la clase real i clasificadas como j

Interpretación

- Diagonal principal (Mii): predicciones correctas
- Elementos fuera de la diagonal (Mij, i≠j): errores de clasificación

Un modelo ideal presenta valores altos en la diagonal y valores cercanos a cero en el resto de la matriz.

La matriz de confusión permite:

- Identificar clases con mayor error
- Detectar patrones de confusión entre categorías
- Analizar el comportamiento del modelo más allá de métricas globales

6.2 Métricas de Evaluación

Las métricas de evaluación permiten medir el rendimiento de un modelo de clasificación supervisada. En problemas multiclase como el presente, no es suficiente utilizar una sola métrica, por lo que se emplea un conjunto de indicadores complementarios.

Accuracy

El Accuracy mide la proporción de predicciones correctas sobre el total de predicciones realizadas:

$$\mathbf{Accuracy} = \frac{\boldsymbol{TP + TN}}{\boldsymbol{TP + TN + \ FP + FN}}$$

En clasificación multiclase, se interpreta como:

$$\mathbf{Accuracy} = \frac{\boldsymbol{Numero\ de\ predicciones\ correctas}}{\boldsymbol{total\ de\ muestras}}$$

Es una métrica global, pero puede ser engañosa si existe desbalance entre clases.

Precision

La precisión mide qué proporción de las predicciones positivas fueron correctas:

$$\mathbf{Precision} = \frac{\boldsymbol{TP}}{\boldsymbol{TP} + \boldsymbol{FP}}$$

Indica la capacidad del modelo para evitar falsos positivos.

Recall (Sensibilidad)

El recall mide la capacidad del modelo para identificar correctamente todas las instancias de una clase:

$$\mathbf{Recall} = \frac{\boldsymbol{TP}}{\boldsymbol{TP} + \boldsymbol{FN}}$$

Es especialmente importante cuando se busca minimizar falsos negativos.

F1-Score

El F1-score es la media armónica entre precision y recall:

$$\mathbf{F1} = 2 * \frac{\boldsymbol{Precision} * \boldsymbol{Recall}}{\boldsymbol{Precision} + \boldsymbol{Recall}}$$

Es útil cuando se requiere un balance entre ambas métricas.

7. Análisis del Modelo Naïve Bayes

7.1 Desempeño General del Modelo

El modelo Naïve Bayes Multinomial implementado obtuvo los siguientes resultados globales:

- Accuracy promedio: 0.9796
- Macro F1-score: 0.9795

La cercanía entre ambas métricas indica un rendimiento equilibrado entre clases, sin sesgo significativo hacia ninguna categoría en particular.

7.2 Interpretación del Modelo

El buen desempeño del modelo se debe a:

- Alta separación léxica entre muchas clases
- Uso efectivo de Bag of Words
- Aplicación de Laplace Smoothing
- Uso de logaritmos para estabilidad numérica

Sin embargo, su principal limitación es la suposición de independencia condicional, lo que impide capturar contexto lingüístico.

7.3 Análisis de la Matriz de Confusión

El análisis de la matriz de confusión muestra que los errores no son aleatorios, sino sistemáticos entre clases semánticamente relacionadas.

Principales confusiones detectadas:

- change_order vs place_order
- get_refund vs cancel_order

Causas principales:

- Uso de vocabulario compartido
- Similitud en la intención del usuario
- Ausencia de contexto en el modelo

Esto es consistente con el enfoque de Naïve Bayes, que se basa únicamente en frecuencia de palabras.

7.4 Desempeño por Tipo de Clase

Clases con alto desempeño

Estas clases presentan F1 cercano a 1.0:

- check_cancellation_fee
- check_payment_methods
- complaint
- contact_customer_service
- contact_human_agent
- newsletter_subscription

Interpretación:
Estas categorías poseen patrones lingüísticos muy específicos y fácilmente diferenciables.

Clases con desempeño medio

- cancel_order
- check_invoice
- delivery_options
- track_order

Interpretación:
Existe cierta superposición de vocabulario entre categorías.

Clases con mayor dificultad

- change_order (~0.91)
- place_order (~0.93)
- get_refund (~0.93)

Causas:

- Alta similitud semántica
- Palabras clave compartidas
- Ambigüedad en las solicitudes

7.5 Estabilidad del Modelo

Mediante validación cruzada con K=5K = 5K=5, se observó:

- Variación mínima entre folds
- Consistencia en los resultados
- Alta capacidad de generalización

Esto indica que el modelo no depende de una partición específica del dataset.

7.6 Conclusión del Análisis

El modelo Naïve Bayes demuestra un desempeño altamente competitivo en tareas de clasificación de texto estructurado.

Sin embargo:

- Sus errores se concentran en clases semánticamente similares
- Su simplicidad limita la comprensión del contexto
- La independencia de palabras es una aproximación fuerte

El modelo cumple satisfactoriamente con el objetivo del proyecto, aunque su mejora requeriría técnicas más avanzadas como modelos basados en contexto o embeddings semánticos.

7. Resultados con K-Folds Cross Validation

Se utilizó validación cruzada con K=5K = 5K=5.

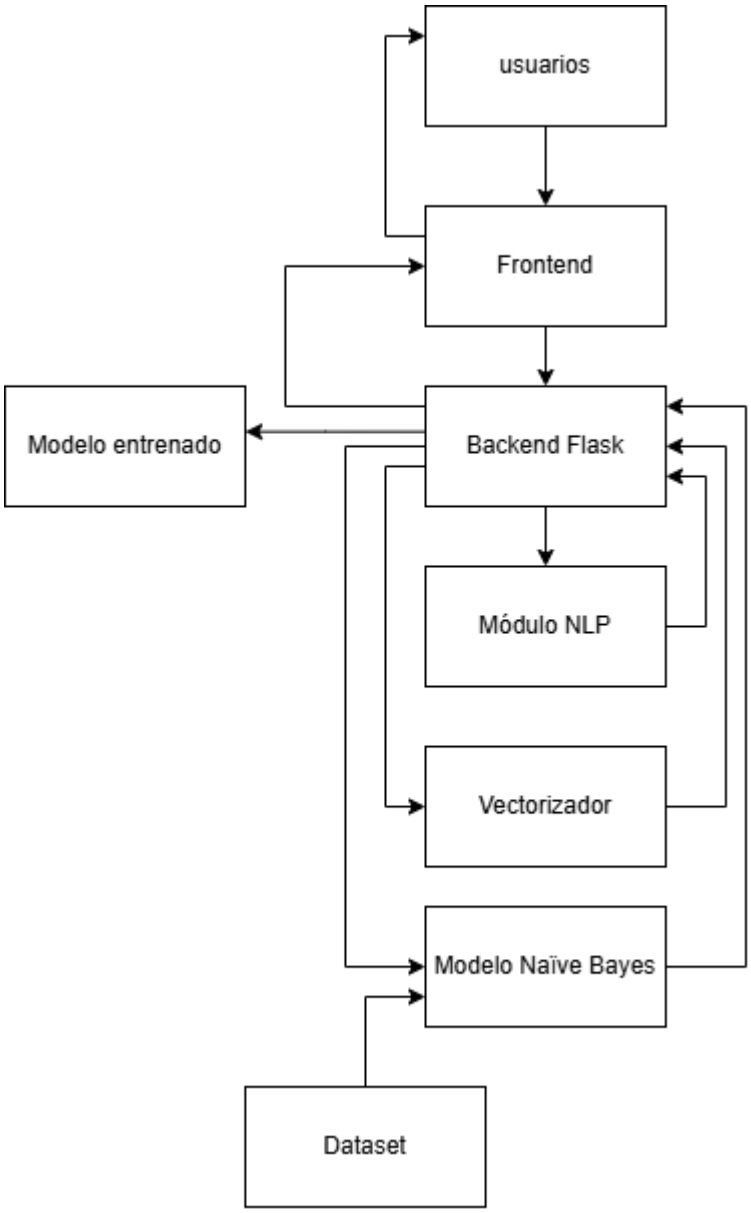
Resultados promedio:

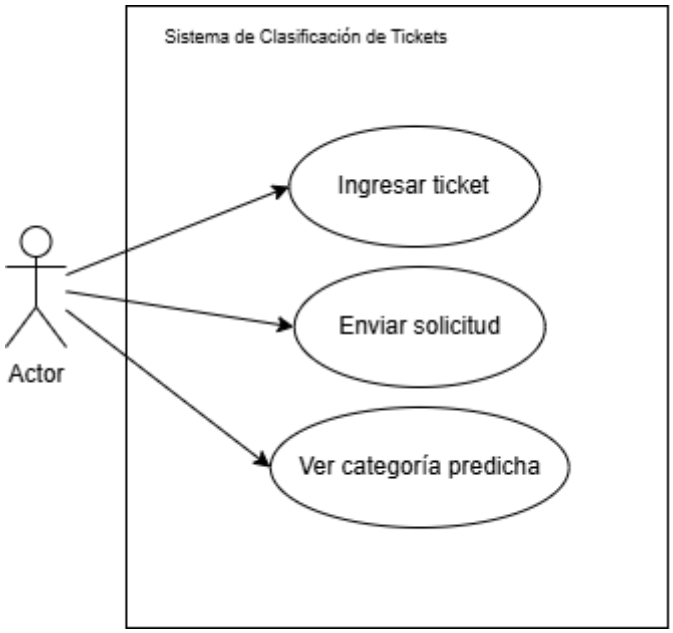
- Accuracy: 0.9796
- Macro F1: 0.9795

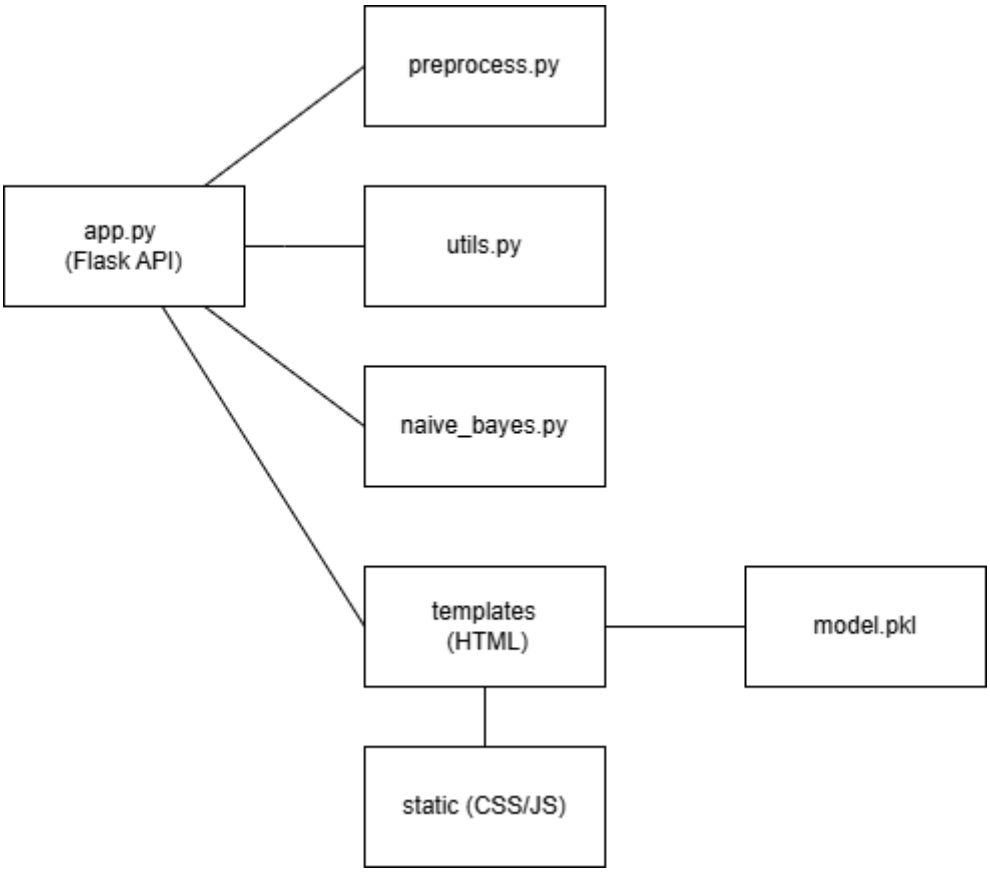
Interpretación

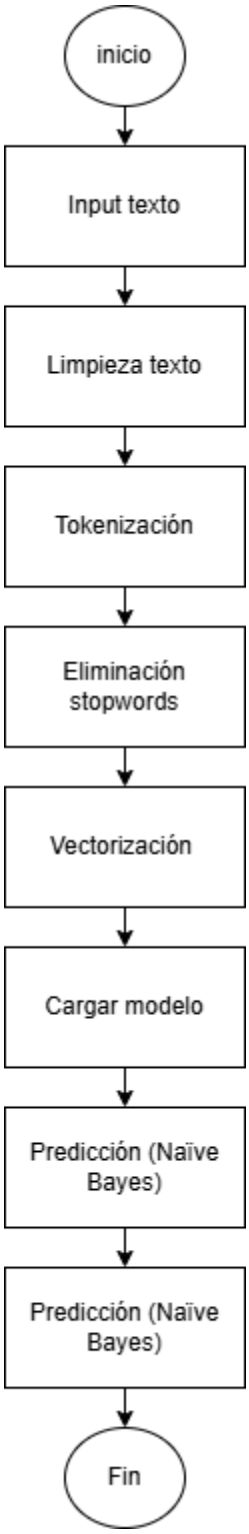
- Alto rendimiento general
- Bajo sesgo entre clases
- Excelente capacidad de generalización
- Variación mínima entre folds (modelo estable)

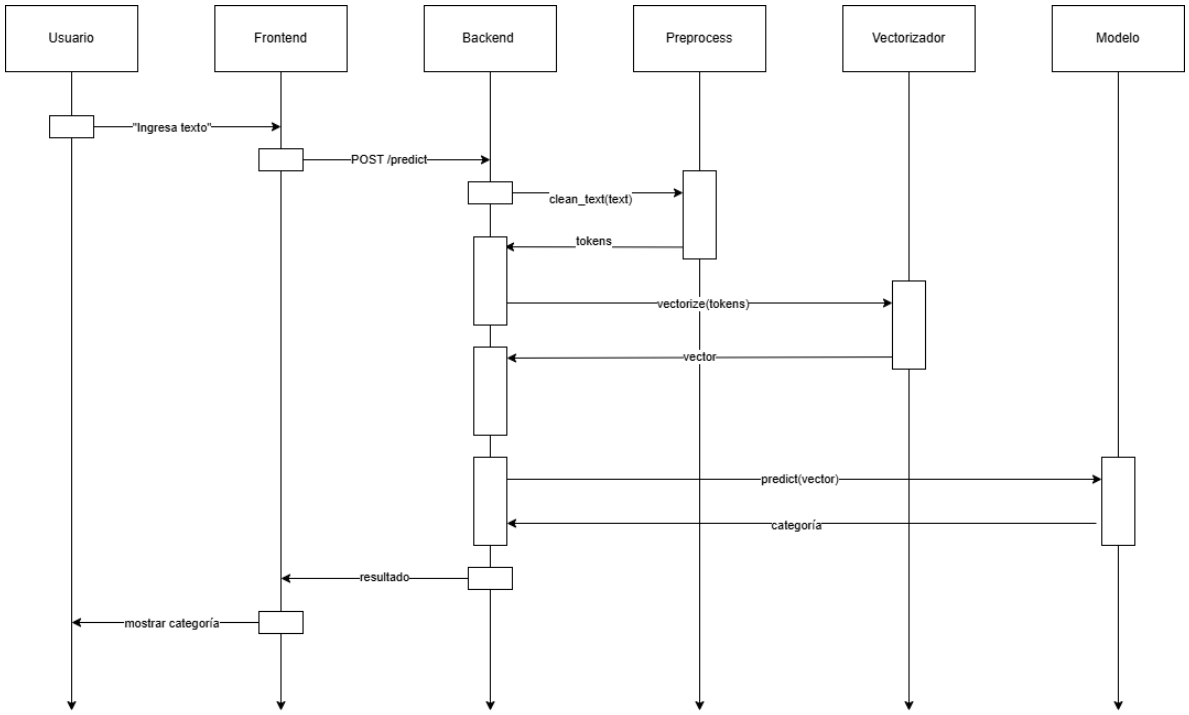
8. Diagramas


usuarios
Frontend
Modelo entrenado
Backend Flask
Módulo NLP
Vectorizador
Modelo Naïve Bayes
Dataset


Sistema de Clasificación de Tickets
Ingresar ticket
Enviar solicitud
Ver categoría predicha
Actor


preprocess.py
app.py
(Flask API)
utils.py
naive_bayes.py
templates
(HTML)
model.pkl
static (CSS/JS)


inicio
Input texto
Limpieza texto
Tokenización
Eliminación stopwords
Vectorización
Cargar modelo
Predicción (Naïve Bayes)
Predicción (Naïve Bayes)
Fin



9. Evidencias de Funcionamiento

A continuación, se presentan las evidencias del funcionamiento del sistema desarrollado, el cual integra un motor de inferencia basado en Naïve Bayes con una aplicación web para la clasificación automática de tickets de soporte.

Se muestra la ejecución del script principal (main.py), donde se realiza:

- Carga del dataset.
- Preprocesamiento del texto.
- Construcción del vocabulario (Bag of Words).
- Entrenamiento del modelo Naïve Bayes.
- Evaluación mediante K-Folds (K=5).
- Cálculo de métricas (Accuracy, Precision, Recall, F1-Score, Macro F1).
- Generación de matrices de confusión.
- Guardado del modelo entrenado (model.pkl).

```python
if "instruction" not in df.columns or "intent" not in df.columns:
    raise Exception("El dataset no tiene las columnas necesarias")

texts = df["instruction"].astype(str)
labels = df["intent"].astype(str)

# K-FOLDS
```

```
==================
Fold 1
==================
Accuracy: 0.9791589132861928

Metrics por clase:
Clase                    Precision  Recall    F1
cancel_order             0.9670     0.9888    0.9778
change_order             0.8511     0.9852    0.9132
change_shipping_address  0.9943     0.9830    0.9886
check_cancellation_fee   1.0000     0.9956    0.9978
check_invoice            0.9854     0.9575    0.9713
check_payment_methods    1.0000     0.9952    0.9976
check_refund_policy      0.9344     0.9956    0.9641
complaint                1.0000     1.0000    1.0000
contact_customer_service 1.0000     0.9950    0.9975
contact_human_agent      0.9945     1.0000    0.9972
create_account           0.9845     0.9845    0.9845
delete_account           0.9823     0.9823    0.9823
delivery_options         0.9765     0.9455    0.9607
delivery_period          0.9836     1.0000    0.9917
edit_account             1.0000     1.0000    1.0000
get_invoice              0.9722     0.9777    0.9789
get_refund               0.9945     0.8667    0.9262
newsletter_subscription  0.9951     1.0000    0.9976
payment_issue            0.9948     0.9948    0.9948
place_order              1.0000     0.8387    0.9123
recover_password         1.0000     1.0000    1.0000
registration_problems    0.9882     0.9709    0.9795
review                   0.9953     1.0000    0.9976
set_up_shipping_address  0.9660     1.0000    0.9827
switch_account           0.9844     0.9947    0.9895
track_order              0.9946     0.9892    0.9919
track_refund             0.9390     0.9950    0.9662

==================
Fold 2
```

```python
if "instruction" not in df.columns or "intent" not in df.columns:
    raise Exception("El dataset no tiene las columnas necesarias")

texts = df["instruction"].astype(str)
labels = df["intent"].astype(str)

# K-FOLDS
```

```
track_refund              0.9390    0.9950    0.9662

===================
Fold 2
===================
Accuracy: 0.9793449944175661

Metrics por clase:
Clase                     Precision  Recall    F1
cancel_order              0.9541     0.9952    0.9742
change_order              0.8717     0.9517    0.9099
change_shipping_address   0.9741     0.9741    0.9741
check_cancellation_fee    0.9944     1.0000    0.9972
check_invoice             0.9864     0.9909    0.9886
check_payment_methods     1.0000     1.0000    1.0000
check_refund_policy       0.9314     0.9896    0.9596
complaint                 0.9952     1.0000    0.9976
contact_customer_service  1.0000     0.9952    0.9976
contact_human_agent       0.9954     1.0000    0.9977
create_account            1.0000     0.9950    0.9975
delete_account            0.9801     0.9949    0.9875
delivery_options          0.9699     0.9253    0.9471
delivery_period           0.9896     0.9896    0.9896
edit_account              0.9698     1.0000    0.9847
get_invoice               0.9953     0.9859    0.9906
get_refund                0.9884     0.8854    0.9341
newsletter_subscription   1.0000     1.0000    1.0000
payment_issue             1.0000     0.9949    0.9975
place_order               1.0000     0.8404    0.9133
recover_password          0.9898     1.0000    0.9949
registration_problems     1.0000     0.9899    0.9949
review                    0.9950     1.0000    0.9975
set_up_shipping_address   0.9387     0.9950    0.9660
switch_account            1.0000     0.9848    0.9924
track_order               0.9892     0.9683    0.9786
track_refund              0.9569     0.9950    0.9756

===================
```

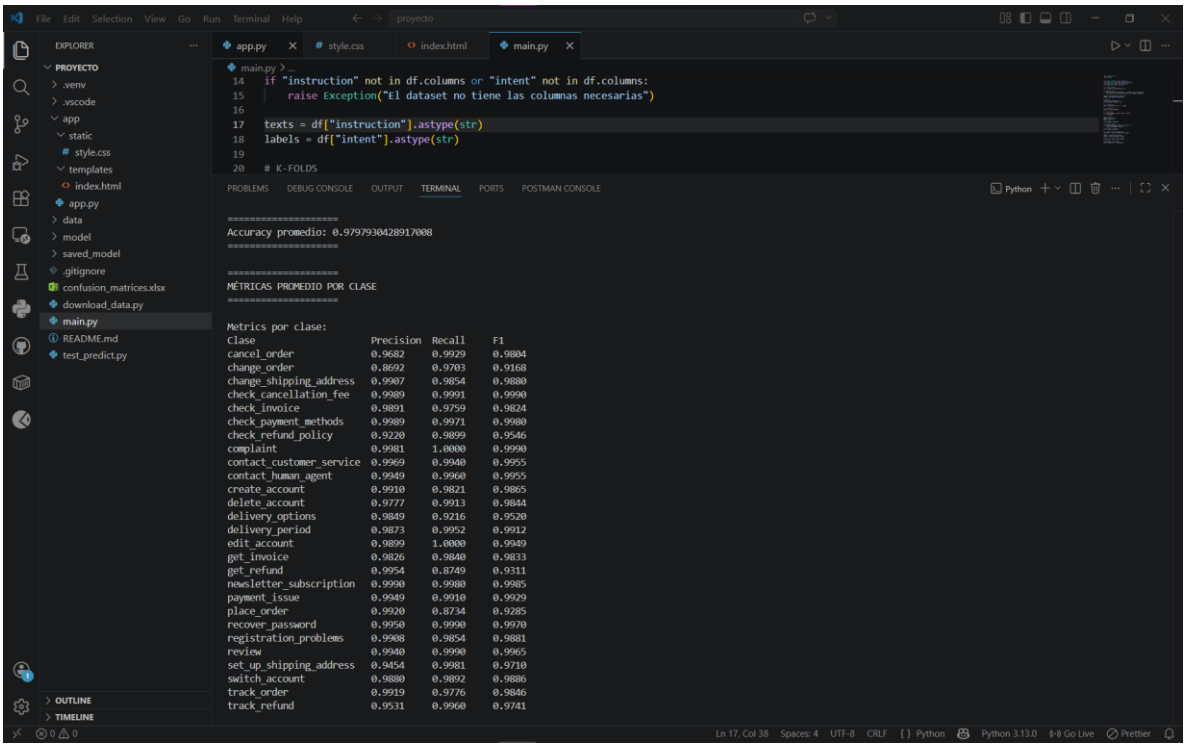
```
===================
Fold 3
===================
Accuracy: 0.9793449944175661

Metrics por clase:
Clase                     Precision  Recall    F1
cancel_order              0.9679     0.9953    0.9814
change_order              0.8500     0.9577    0.9007
change_shipping_address   0.9898     0.9848    0.9873
check_cancellation_fee    1.0000     1.0000    1.0000
check_invoice             1.0000     0.9674    0.9834
check_payment_methods     1.0000     1.0000    1.0000
check_refund_policy       0.9330     0.9799    0.9559
complaint                 1.0000     1.0000    1.0000
contact_customer_service  0.9898     0.9949    0.9924
contact_human_agent       0.9951     0.9902    0.9926
create_account            0.9945     0.9837    0.9891
delete_account            0.9796     0.9897    0.9846
delivery_options          0.9888     0.9167    0.9514
delivery_period           0.9783     0.9912    0.9847
edit_account              0.9895     1.0000    0.9947
get_invoice               0.9747     0.9948    0.9847
get_refund                1.0000     0.9000    0.9474
newsletter_subscription   1.0000     1.0000    1.0000
payment_issue             0.9950     0.9950    0.9950
place_order               0.9763     0.8684    0.9192
recover_password          0.9951     0.9951    0.9951
registration_problems     0.9950     0.9950    0.9950
review                    0.9948     1.0000    0.9974
set_up_shipping_address   0.9607     0.9955    0.9778
switch_account            0.9903     0.9951    0.9927
track_order               0.9860     0.9505    0.9679
track_refund              0.9621     0.9951    0.9783

===================
Fold 4
===================
```

```python
if "instruction" not in df.columns or "intent" not in df.columns:
    raise Exception("El dataset no tiene las columnas necesarias")

texts = df["instruction"].astype(str)
labels = df["intent"].astype(str)

# K-FOLDS
```

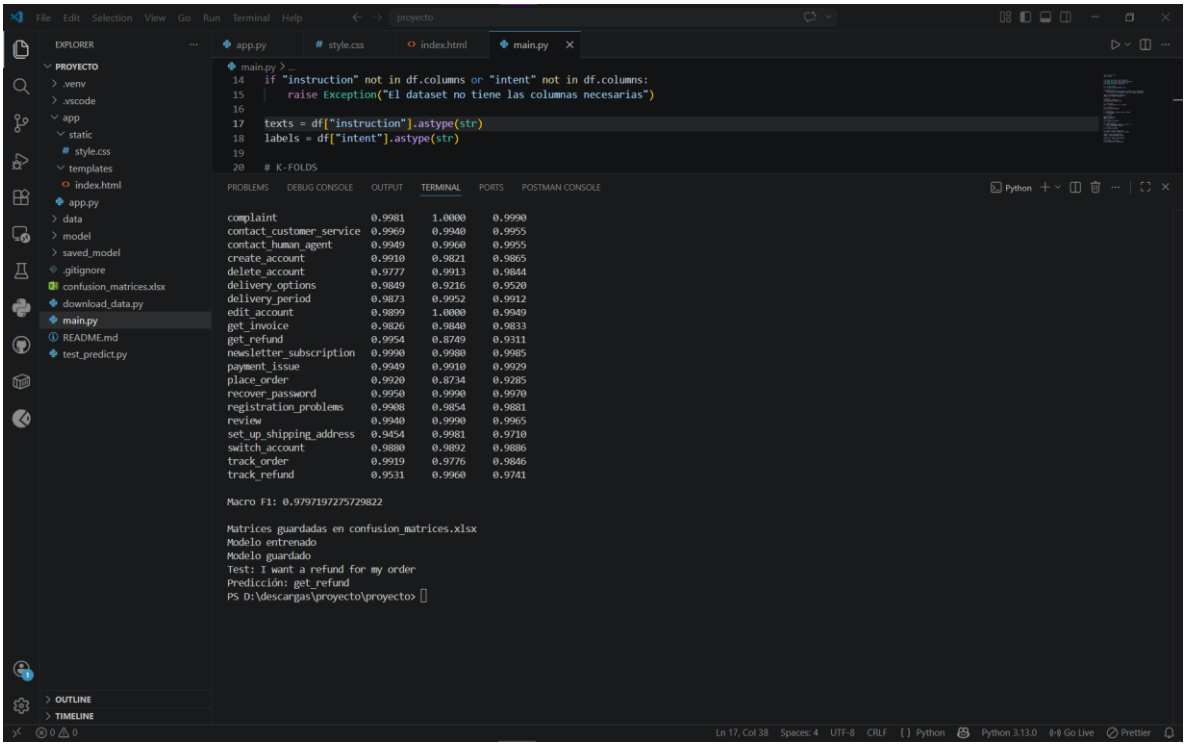
```
==================
Fold 4
==================
Accuracy: 0.980647562337179

Metrics por clase:
Clase                    Precision  Recall    F1
cancel_order             0.9842     0.9947    0.9894
change_order             0.8791     0.9844    0.9287
change_shipping_address  1.0000     0.9950    0.9975
check_cancellation_fee   1.0000     1.0000    1.0000
check_invoice            0.9895     0.9844    0.9869
check_payment_methods    0.9946     1.0000    0.9973
check_refund_policy      0.8835     0.9945    0.9357
complaint                1.0000     1.0000    1.0000
contact_customer_service 1.0000     0.9904    0.9952
contact_human_agent      0.9896     0.9948    0.9922
create_account           0.9858     0.9858    0.9858
delete_account           0.9853     0.9950    0.9901
delivery_options         0.9947     0.8863    0.9373
delivery_period          0.9950     1.0000    0.9975
edit_account             0.9955     1.0000    0.9977
get_invoice              0.9847     0.9847    0.9847
get_refund               0.9940     0.8384    0.9096
newsletter_subscription  1.0000     0.9948    0.9974
payment_issue            1.0000     0.9952    0.9976
place_order              0.9888     0.9124    0.9491
recover_password         1.0000     1.0000    1.0000
registration_problems    0.9913     0.9913    0.9913
review                   0.9948     1.0000    0.9974
set_up_shipping_address  0.9306     1.0000    0.9640
switch_account           0.9856     0.9763    0.9810
track_order              1.0000     0.9903    0.9951
track_refund             0.9524     0.9950    0.9732

==================
Fold 5
```


```python
if "instruction" not in df.columns or "intent" not in df.columns:
    raise Exception("El dataset no tiene las columnas necesarias")

texts = df["instruction"].astype(str)
labels = df["intent"].astype(str)

# K-FOLDS
```


```
track_refund             0.9524     0.9950    0.9732

==================
Fold 5
==================
Accuracy: 0.98046875

Metrics por clase:
Clase                    Precision  Recall    F1
cancel_order             0.9676     0.9905    0.9789
change_order             0.8939     0.9725    0.9316
change_shipping_address  0.9951     0.9903    0.9927
check_cancellation_fee   1.0000     1.0000    1.0000
check_invoice            0.9844     0.9793    0.9818
check_payment_methods    1.0000     0.9904    0.9952
check_refund_policy      0.9275     0.9897    0.9576
complaint                0.9953     1.0000    0.9977
contact_customer_service 0.9947     0.9947    0.9947
contact_human_agent      1.0000     0.9952    0.9976
create_account           0.9900     0.9614    0.9755
delete_account           0.9613     0.9943    0.9775
delivery_options         0.9946     0.9343    0.9635
delivery_period          0.9901     0.9950    0.9926
edit_account             0.9947     1.0000    0.9974
get_invoice              0.9860     0.9770    0.9815
get_refund               1.0000     0.8841    0.9385
newsletter_subscription  1.0000     0.9950    0.9975
payment_issue            0.9848     0.9749    0.9798
place_order              0.9949     0.9070    0.9489
recover_password         0.9903     1.0000    0.9951
registration_problems    0.9796     0.9796    0.9796
review                   0.9902     0.9951    0.9926
set_up_shipping_address  0.9312     1.0000    0.9644
switch_account           0.9798     0.9949    0.9873
track_order              0.9895     0.9895    0.9895
track_refund             0.9550     1.0000    0.9770

==================
```

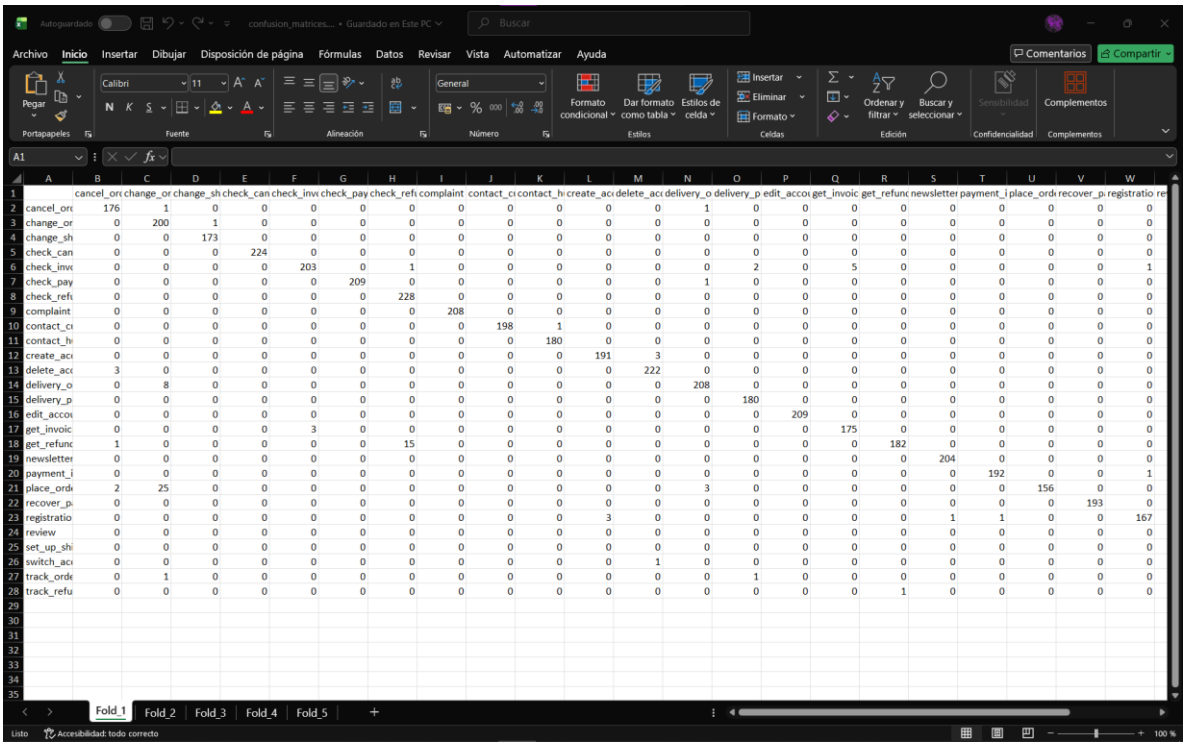


Validación del Modelo con Prueba Controlada
Se incluye una prueba directa del modelo con un texto de ejemplo:
test_text = "I want a refund for my order"
El sistema procesa el texto y devuelve la categoría predicha.

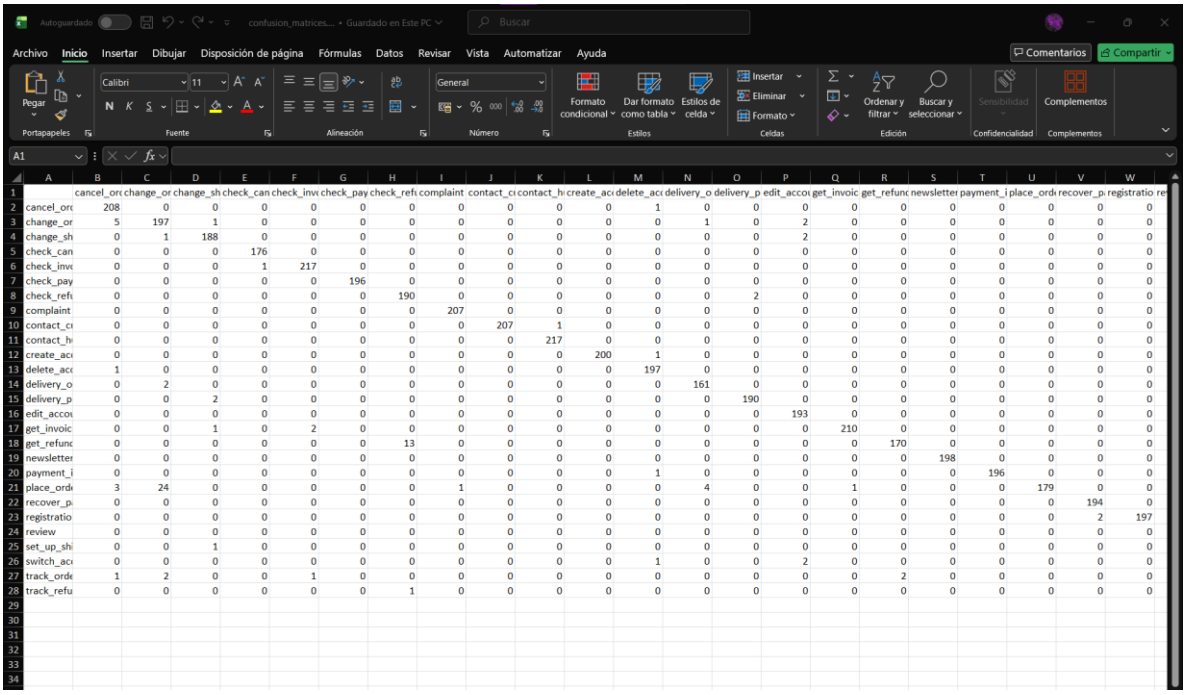


Generación de Matrices de Confusión

Durante la validación con K-Folds, se generan matrices de confusión para cada fold, las cuales permiten analizar el desempeño del modelo por clase.
Adicionalmente, estas matrices son exportadas a un archivo Excel: confusion_matrices.xlsx



	cancel_or	change_or	change_sh	check_can	check_inv	check_pay	check_refu	complaint	contact_cu	contact_h	create_acc	delete_acc	delivery_o	delivery_p	edit_accou	get_invoic	get_refunc	newsletter	payment_i	place_ord	recover_p	registratio	re
cancel_or	176	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	
change_or	0	200	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
change_sh	0	0	173	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
check_can	0	0	0	224	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
check_inv	0	0	0	0	203	0	1	0	0	0	0	0	0	2	0	5	0	0	0	0	0	1	
check_pay	0	0	0	0	0	209	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	
check_refu	0	0	0	0	0	0	228	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
complaint	0	0	0	0	0	0	0	208	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
contact_cu	0	0	0	0	0	0	0	0	198	1	0	0	0	0	0	0	0	0	0	0	0	0	
contact_h	0	0	0	0	0	0	0	0	0	180	0	0	0	0	0	0	0	0	0	0	0	0	
create_acc	0	0	0	0	0	0	0	0	0	0	191	3	0	0	0	0	0	0	0	0	0	0	
delete_acc	3	0	0	0	0	0	0	0	0	0	0	222	0	0	0	0	0	0	0	0	0	0	
delivery_o	0	8	0	0	0	0	0	0	0	0	0	0	208	0	0	0	0	0	0	0	0	0	
delivery_p	0	0	0	0	0	0	0	0	0	0	0	0	0	180	0	0	0	0	0	0	0	0	
edit_accou	0	0	0	0	0	0	0	0	0	0	0	0	0	0	209	0	0	0	0	0	0	0	
get_invoic	0	0	0	0	3	0	0	0	0	0	0	0	0	0	0	175	0	0	0	0	0	0	
get_refunc	1	0	0	0	0	0	15	0	0	0	0	0	0	0	0	0	182	0	0	0	0	0	
newsletter	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	204	0	0	0	0	
payment_i	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	192	0	0	1	
place_ord	2	25	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	156	0	0	
recover_p	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	193	0	
registratio	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	1	1	0	0	167	
review	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
set_up_shi	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
switch_acc	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	
track_orde	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
track_refu	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	



	cancel_or	change_or	change_sh	check_can	check_inv	check_pay	check_refu	complaint	contact_cu	contact_h	create_acc	delete_acc	delivery_o	delivery_p	edit_accou	get_invoic	get_refunc	newsletter	payment_i	place_ord	recover_p	registratio	re
cancel_or	208	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	
change_or	5	197	1	0	0	0	0	0	0	0	0	0	1	0	2	0	0	0	0	0	0	0	
change_sh	0	1	188	0	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0	
check_can	0	0	0	176	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
check_inv	0	0	0	1	217	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
check_pay	0	0	0	0	0	196	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
check_refu	0	0	0	0	0	0	190	0	0	0	0	0	0	2	0	0	0	0	0	0	0	0	
complaint	0	0	0	0	0	0	0	207	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
contact_cu	0	0	0	0	0	0	0	0	207	1	0	0	0	0	0	0	0	0	0	0	0	0	
contact_h	0	0	0	0	0	0	0	0	0	217	0	0	0	0	0	0	0	0	0	0	0	0	
create_acc	0	0	0	0	0	0	0	0	0	0	200	1	0	0	0	0	0	0	0	0	0	0	
delete_acc	1	0	0	0	0	0	0	0	0	0	0	197	0	0	0	0	0	0	0	0	0	0	
delivery_o	0	2	0	0	0	0	0	0	0	0	0	0	161	0	0	0	0	0	0	0	0	0	
delivery_p	0	0	2	0	0	0	0	0	0	0	0	0	0	190	0	0	0	0	0	0	0	0	
edit_accou	0	0	0	0	0	0	0	0	0	0	0	0	0	0	193	0	0	0	0	0	0	0	
get_invoic	0	0	1	0	2	0	0	0	0	0	0	0	0	0	0	210	0	0	0	0	0	0	
get_refunc	0	0	0	0	0	0	13	0	0	0	0	0	0	0	0	0	170	0	0	0	0	0	
newsletter	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	198	0	0	0	0	
payment_i	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	196	0	0	0	
place_ord	3	24	0	0	0	0	0	1	0	0	0	0	4	0	0	1	0	0	0	179	0	0	
recover_p	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	194	0	
registratio	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	197	
review	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
set_up_shi	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
switch_acc	0	0	0	0	0	0	0	0	0	0	0	1	0	0	2	0	0	0	0	0	0	0	
track_orde	1	2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	
track_refu	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Interfaz Web en Funcionamiento

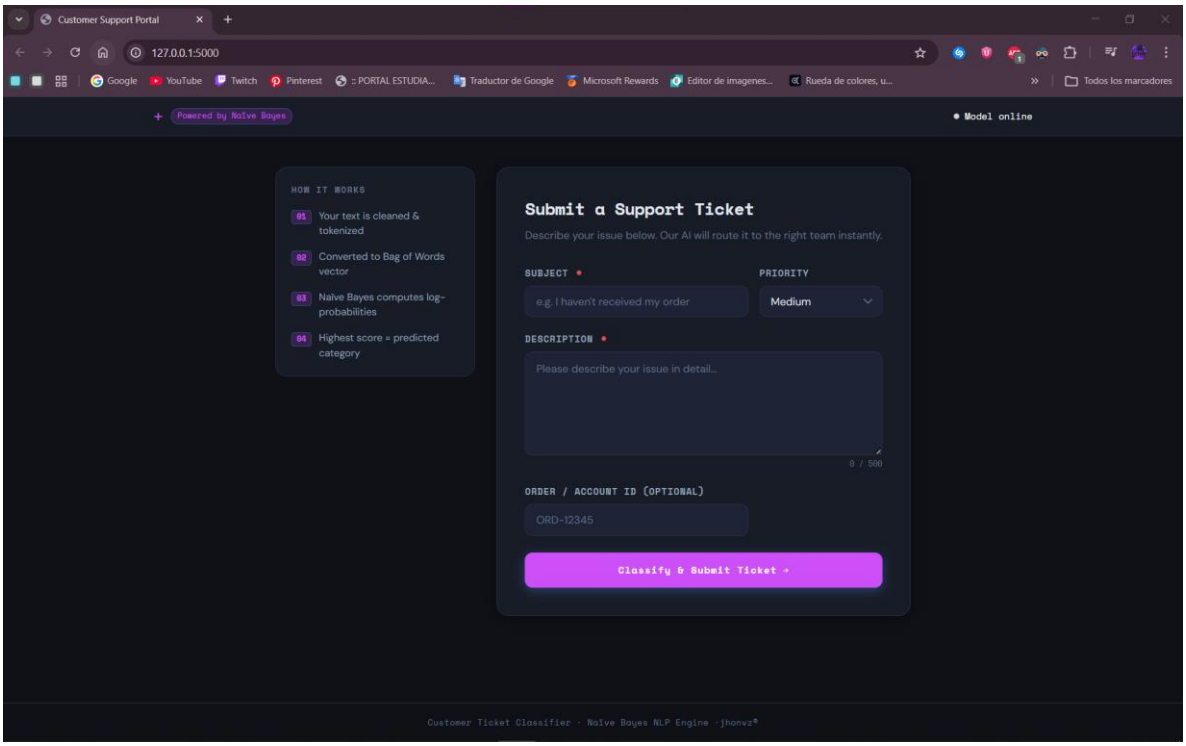
Se desarrolló una aplicación web utilizando Flask que simula un sistema de tickets de soporte.

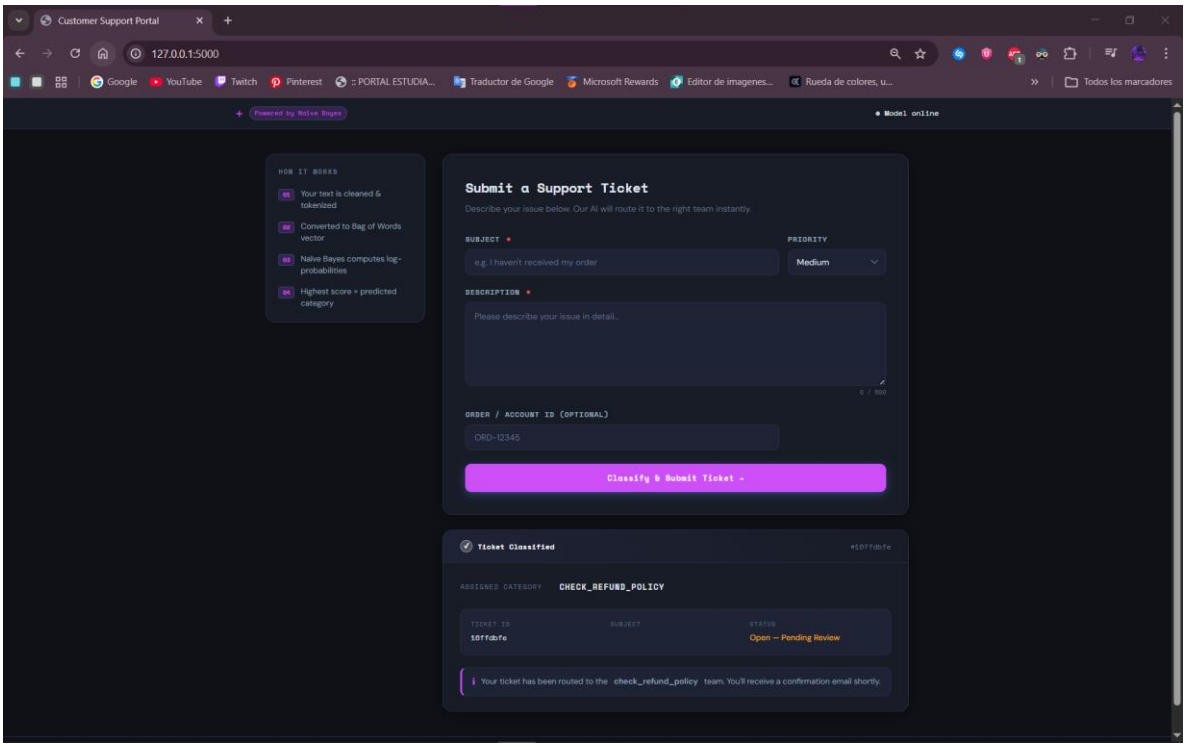
El usuario puede:

- Ingresar un **Subject** (resumen del problema).
- Ingresar una **Description** (detalle del problema).
- Enviar la solicitud para clasificación.

El sistema responde con:

- ID de ticket generado automáticamente.
- Categoría predicha por el modelo.





Flujo Completo del Sistema

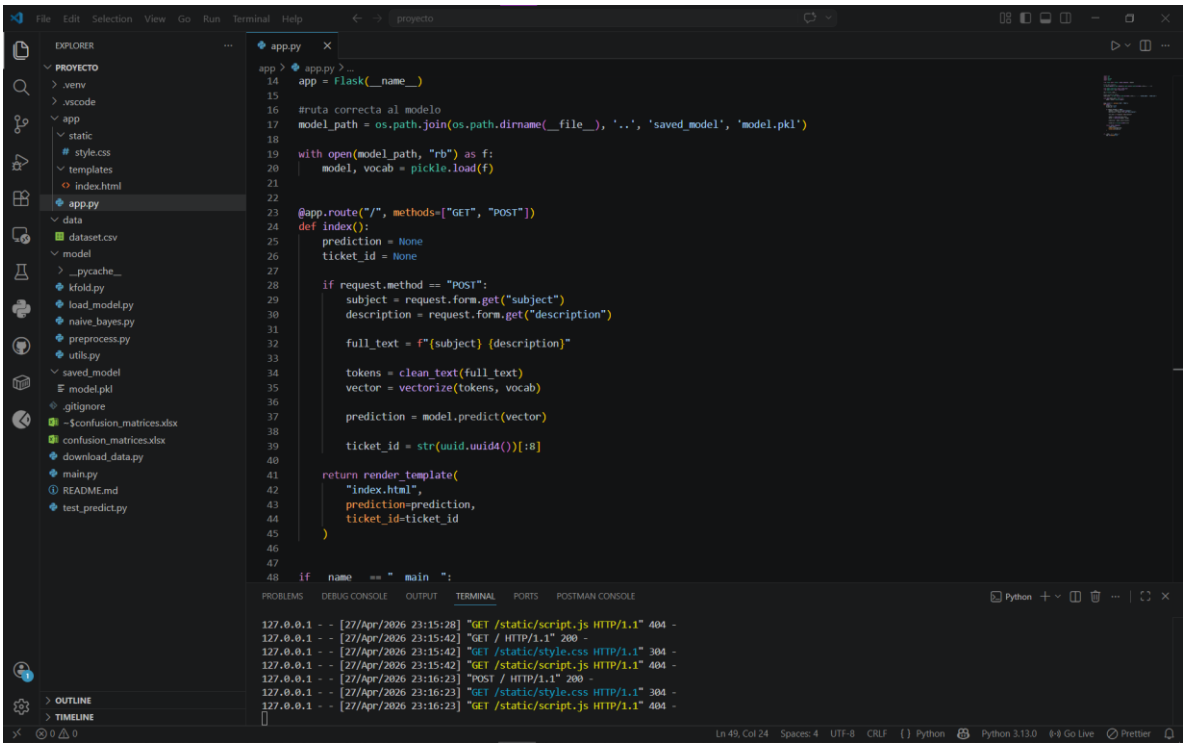
Se valida la integración completa del sistema:

1. Usuario ingresa datos en la interfaz web.
2. El frontend envía la solicitud al backend.
3. El backend:
    - Limpia el texto (preprocesamiento).
    - Vectoriza el texto.
    - Aplica el modelo Naïve Bayes.
4. Se retorna la predicción al frontend.
5. Se muestra el resultado al usuario.

Estructura del Proyecto

Se incluye la organización del proyecto, evidenciando la separación de responsabilidades:

- model/: implementación del modelo y utilidades.
- data/: dataset utilizado.
- saved_model/: modelo entrenado.
- app/: aplicación web (Flask).
- main.py: entrenamiento y evaluación.



Código Fuente Relevante

Se destacan las partes clave del sistema:

- Implementación de Naïve Bayes.
- Preprocesamiento de texto.
- Vectorización (Bag of Words).
- K-Folds manual.
- Integración con Flask.

File Edit Selection View Go Run Terminal Help proyecto

EXPLORER

- PROYECTO
  - .venv
  - .vscode
  - app
    - static
      - style.css
    - templates
      - index.html
    - app.py
  - data
    - dataset.csv
  - model
    - __pycache__
    - kfold.py
    - load_model.py
    - naive_bayes.py
    - preprocess.py
    - utils.py
  - saved_model
    - model.pkl
  - .gitignore
  - ~$confusion_matrices.xlsx
  - confusion_matrices.xlsx
  - download_data.py
  - main.py
  - README.md
  - test_predict.py

app.py | naive_bayes.py

model > naive_bayes.py > ...

```python
class NaiveBayes:
        self.classes = []

    def train(self, X, y):
        self.classes = set(y)
        self.vocab_size = len(X[0])

        class_counts = defaultdict(int)
        word_counts = {c: [0]*self.vocab_size for c in self.classes}
        total_words = {c: 0 for c in self.classes}

        # contar
        for i in range(len(X)):
            c = y[i]
            class_counts[c] += 1

            for j in range(self.vocab_size):
                word_counts[c][j] += X[i][j]
                total_words[c] += X[i][j]

        total_docs = len(y)

        # prior P(clase)
        for c in self.classes:
            self.class_probs[c] = math.log(class_counts[c] / total_docs)

        # likelihood con Laplace
        for c in self.classes:
            self.word_probs[c] = []
            for j in range(self.vocab_size):
                prob = (word_counts[c][j] + 1) / (total_words[c] + self.vocab_size)
                self.word_probs[c].append(math.log(prob))

    def predict(self, x):
        scores = {}
```

PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS POSTMAN CONSOLE

```
127.0.0.1 - - [27/Apr/2026 23:15:28] "GET /static/script.js HTTP/1.1" 404 -
127.0.0.1 - - [27/Apr/2026 23:15:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [27/Apr/2026 23:15:42] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:15:42] "GET /static/script.js HTTP/1.1" 404 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/script.js HTTP/1.1" 404 -
```

OUTLINE

TIMELINE

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.13.0 Go Live Prettier

File Edit Selection View Go Run Terminal Help proyecto

EXPLORER

- PROYECTO
  - .venv
  - .vscode
  - app
    - static
      - style.css
    - templates
      - index.html
    - app.py
  - data
    - dataset.csv
  - model
    - __pycache__
    - kfold.py
    - load_model.py
    - naive_bayes.py
    - preprocess.py
    - utils.py
  - saved_model
    - model.pkl
  - .gitignore
  - ~$confusion_matrices.xlsx
  - confusion_matrices.xlsx
  - download_data.py
  - main.py
  - README.md
  - test_predict.py

app.py | naive_bayes.py

model > naive_bayes.py > ...

```python
class NaiveBayes:
    def train(self, X, y):
        # likelihood con Laplace
        for c in self.classes:
            self.word_probs[c] = []
            for j in range(self.vocab_size):
                prob = (word_counts[c][j] + 1) / (total_words[c] + self.vocab_size)
                self.word_probs[c].append(math.log(prob))

    def predict(self, x):
        scores = {}
```


```python
        for c in self.classes:
            score = self.class_probs[c]

            for i in range(len(x)):
                score += x[i] * self.word_probs[c][i]

            scores[c] = score

        return max(scores, key=scores.get)
```

PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS POSTMAN CONSOLE

```
127.0.0.1 - - [27/Apr/2026 23:15:28] "GET /static/script.js HTTP/1.1" 404 -
127.0.0.1 - - [27/Apr/2026 23:15:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [27/Apr/2026 23:15:42] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:15:42] "GET /static/script.js HTTP/1.1" 404 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/script.js HTTP/1.1" 404 -
```

OUTLINE

TIMELINE

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.13.0 Go Live Prettier

```python
# ========================
# K-FOLDS
# ========================
def run_kfold(texts, labels, k=5):
    folds = k_folds_split(texts, labels, k)

    accuracies = []
    all_matrices = []

    # acumulador métricas
    metrics_acum = defaultdict(lambda: {"precision": 0, "recall": 0, "f1": 0})

    for i in range(k):
        print(f"\n====================")
        print(f"Fold {i+1}")
        print(f"====================")

        test_data = folds[i]
        train_data = [item for j in range(k) if j != i for item in folds[j]]

        train_texts = [x for x, y in train_data]
        train_labels = [y for x, y in train_data]

        test_texts = [x for x, y in test_data]
        test_labels = [y for x, y in test_data]

        # PREPROCESAMIENTO
        train_docs = [clean_text(t) for t in train_texts]
        test_docs = [clean_text(t) for t in test_texts]

        # VOCABULARIO (solo train)
        vocab = build_vocab(train_docs)

        # VECTORIZACIÓN
        X_train = [vectorize(doc, vocab) for doc in train_docs]
        X_test = [vectorize(doc, vocab) for doc in test_docs]
```

```
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/script.js HTTP/1.1" 404 -
```

```python
def run_kfold(texts, labels, k=5):
        # PREPROCESAMIENTO
        train_docs = [clean_text(t) for t in train_texts]
        test_docs = [clean_text(t) for t in test_texts]

        # VOCABULARIO (solo train)
        vocab = build_vocab(train_docs)

        # VECTORIZACIÓN
        X_train = [vectorize(doc, vocab) for doc in train_docs]
        X_test = [vectorize(doc, vocab) for doc in test_docs]

        # MODELO
        model = NaiveBayes()
        model.train(X_train, train_labels)

        # PREDICCIÓN
        predictions = [model.predict(x) for x in X_test]

        # ACCURACY
        acc = evaluate(test_labels, predictions)
        accuracies.append(acc)
        print("Accuracy:", acc)

        # MATRIZ
        matrix, classes = build_confusion_matrix(test_labels, predictions)
        all_matrices.append((matrix, classes))

        # MÉTRICAS
        metrics = compute_metrics(matrix, classes)
        print_metrics(metrics)

        # ACUMULAR
        for c in metrics:
            metrics_acum[c]["precision"] += metrics[c]["precision"]
            metrics_acum[c]["recall"] += metrics[c]["recall"]
            metrics_acum[c]["f1"] += metrics[c]["f1"]

    # ========================
```


```
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/script.js HTTP/1.1" 404 -
```

```python
import sys
import os
import pickle
import uuid

from flask import Flask, render_template, request

#(root del proyecto)
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from model.preprocess import clean_text
from model.utils import vectorize

app = Flask(__name__)

#ruta correcta al modelo
model_path = os.path.join(os.path.dirname(__file__), '..', 'saved_model', 'model.pkl')

with open(model_path, "rb") as f:
    model, vocab = pickle.load(f)


@app.route("/", methods=["GET", "POST"])
def index():
    prediction = None
    ticket_id = None

    if request.method == "POST":
        subject = request.form.get("subject")
        description = request.form.get("description")

        full_text = f"{subject} {description}"

        tokens = clean_text(full_text)
        vector = vectorize(tokens, vocab)

        prediction = model.predict(vector)

        ticket_id = str(uuid.uuid4())[:8]
```

```
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/script.js HTTP/1.1" 404 -
```


```python
@app.route("/", methods=["GET", "POST"])
def index():
    prediction = None
    ticket_id = None

    if request.method == "POST":
        subject = request.form.get("subject")
        description = request.form.get("description")

        full_text = f"{subject} {description}"

        tokens = clean_text(full_text)
        vector = vectorize(tokens, vocab)

        prediction = model.predict(vector)

        ticket_id = str(uuid.uuid4())[:8]
```


```python
    return render_template(
        "index.html",
        prediction=prediction,
        ticket_id=ticket_id
    )


if __name__ == "__main__":
    app.run(debug=True)
```


```
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [27/Apr/2026 23:16:23] "GET /static/script.js HTTP/1.1" 404 -
```

10. Conclusión

El modelo Naïve Bayes Multinomial implementado demuestra ser altamente eficiente para la clasificación de texto, alcanzando un rendimiento cercano al 98%.

Sin embargo, su principal limitación radica en la suposición de independencia entre palabras, lo cual dificulta la diferenciación entre clases con contexto similar.

Posibles mejoras:

- Uso de n-gramas para capturar contexto
- Representaciones más avanzadas (TF-IDF, embeddings)
- Modelos más complejos (SVM, redes neuronales)

A pesar de estas limitaciones, el sistema cumple satisfactoriamente con los objetivos planteados y valida la efectividad de técnicas probabilísticas en tareas de NLP.